

SpartanGold++

Joseph Bucholtz, Joshua Krone

San Jose State University

joseph.bucholtz@sjsu.edu

joshua.krone@sjsu.edu

Abstract— This paper presents a C++ port of the blockchain system SpartanGold. SpartanGold was originally developed in JavaScript. Our objective is to replicate the functionality of it in C++ to improve performance and expand our coding capabilities. The system we created successfully reproduced SpartanGold blockchain components such as transactions, blocks, and mining logic, all inside a single-process environment. Also, miners finish with more coins with the same parameters as the JS version as our project is more performant. However, interoperability and multi-instance was not reached.

I. INTRODUCTION

SpartanGold is a simplified blockchain platform that was designed for educational use within our blockchain and crypto class. Unlike Bitcoin, it has an account-based model and excludes scripting to help students understand Blockchain technology.

The original implementation is written in JavaScript, which allows for rapid prototyping and accessibility. This is great for learning in a class, but we wanted to expand into a different system. Lower-level languages such as C/C++ give massive advantages in performance, memory control,

and type safety which makes it a lot more suitable for high speed, production-level systems.

The goal of this project is to create SpartanGold++, a port of SpartanGold to C++. We wanted to make sure that while porting it, it maintains functional equivalence with the JavaScript implementation. In addition, our last goal is to set up multi-instance and interoperability between the two projects.

II. TIMELINE

The original project timeline was over these four structured phases:

- **Week 1:** Project setup using CMake and initial implementation of core components.
- **Week 2:** Completion of core blockchain logic and establishment of a functional single-threaded system.
- **Week 3:** Develop asynchronous instances.
- **Week 4:** Test the system and check for interoperability.

This proved to be an overestimation of our abilities and the final timeline involved cutting out asynchronous instances and interoperability. We found that interoperability was not going to occur given

the implementation for the hashing library in C++ was too different. Transactions would always be rejected. In terms of multi-instance, the porting of the project took more time.

THE FINAL TIMELINE

Week 1

We set up the project using CMake and made decisions on the project design/architecture. We had three iterations of the design, where the third design was what we finally implemented.

Week 2

Completed core components such as utils, transaction, and block. This week, we figured out what libraries we would need, how to be explicit with our object definitions, and found a reproducible way to implement the next classes.

Week 3

Developed the final components such as the network, miner/client, and blockchain.

Week 4

Tested our system, fixed bugs, and handled any parts that needed further development.

III. CHALLENGES

Throughout the project we encountered a handful of challenges:

A. Language and Paradigm Differences

Since Javascript is a dynamic typed language whereas C++ is a static typed language, we needed to redesign data structures and adapt the system logic to fit a stricter programming model.

B. Library and Tooling Differences

Identifying C++ libraries for cryptography and JSON brought a lot of complexity, as direct equivalents to JavaScript libraries is not straightforward.

C. Concurrency and Networking

Implementing asynchronous behavior and multi-threading was difficult in the timeframe we allocated to ourselves. When we realized that the single-threaded mode was taking significantly longer than expected, we stopped thinking about concurrency and moved to getting a working project.

D. Interoperability

After deciding to not develop a concurrent system, achieving compatibility between JavaScript and C++ was also not going to occur. While concurrent programming stopped us in this category, we would have faced challenges like transactions not counting and network protocol issues. The reason being that to design a C++ style of this project it required changing too many small factors in the original that would fail when communicating with each other.

IV. RESULTS

The SpartanGold++ implementation successfully replicates the main, single-instance mode functionality of the original SpartanGold.

The following components were fully integrated:

- Block creation and validation
- Transaction processing
- Supporting utility functions

In addition, we also managed to achieve implementing a fully functional single-process blockchain system, including:

- Client and miner behavior
- Blockchain state management
- Network simulation within a single process

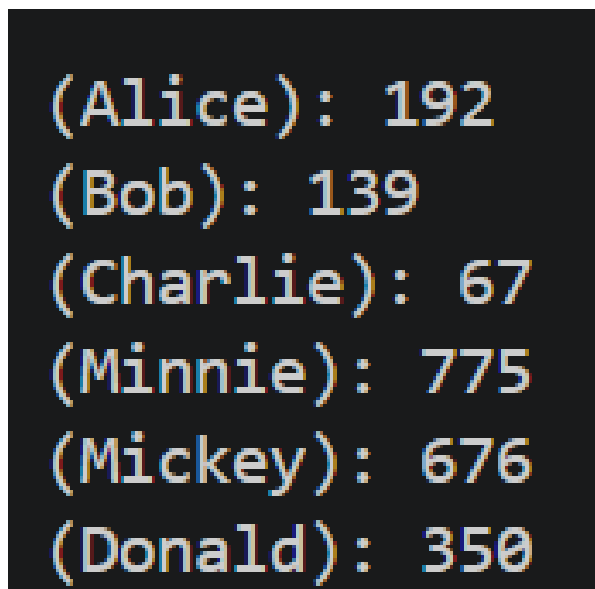
Single-Mode

The completion of single-process mode is proof that the system correctly implements essential blockchain functions like proof-of-work mining, transaction validation, and block propagation inside a controlled environment.

Performance

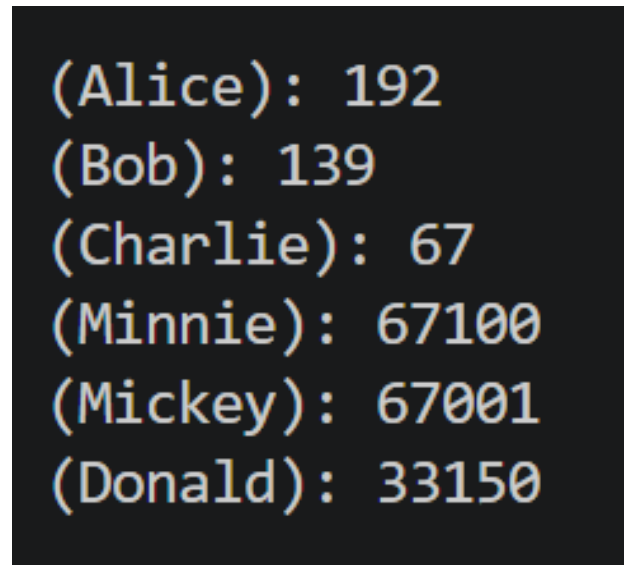
We found that under the same parameters the miners have more coins than in the JS version.

ORIGINAL PROJECT (8secs)



```
(Alice): 192
(Bob): 139
(Charlie): 67
(Minnie): 775
(Mickey): 676
(Donald): 350
```

OUR PROJECT (8secs)



```
(Alice): 192
(Bob): 139
(Charlie): 67
(Minnie): 67100
(Mickey): 67001
(Donald): 33150
```

This is because even with the same mining power, same reward, and same BigInt PoW target, our version is more performant. Our miners can mine blocks faster and be rewarded more. This was the main goal of switching languages and our project correctly demonstrates this.

Limitations

Despite these accomplishments, full interoperability with the JavaScript implementation was not achieved and support for multi-process and asynchronous networking remains future objectives as the time constraint only allowed for so much.

V. CONCLUSION

This project demonstrates the semi-successful porting of SpartanGold in JavaScript to C++. SpartanGold++ also preserves the fundamental behavior of the original system while gaining the benefits of a statically typed and high-performance language. A key outcome from this project

is being able to successfully implement a complete single-process blockchain system.

The primary limitation of the project is not being able to complete interoperability. This shows the complexity when it comes to cross-language distributed systems.

Future work on this project may include completing interoperability features, adding multi-process environments, and implementing asynchronous networking.